DecisionGuard: SME Finance Copilot. Technical Documentation

Deliverable 01: system architecture, core algorithms and security design

Item	Detail
Document	Technical Documentation, version 1.0
Date	20 October 2026
Competition	2026 Shenzhen International FinTech Competition (FinTechathon), International Track
Topic	**E: SME Finance Copilot** (financing-option matching, cash-flow management with alerts, business financial analysis)
Team	DecisionGuard: Md. Amdadul Haque (team lead), Nobo Ishraque, Labib Afnan, Adil Akhtab Abir
University	Bangladesh University of Professionals (BUP), Dhaka, Bangladesh
Code	`main` branch of the submitted repository, at the code-freeze commit recorded in `docs/submission/evidence/README.md`
Companion documents	Security Self-Assessment (`docs/submission/security-self-assessment.md`); Execution Evidence (`docs/submission/evidence/`)

Abstract

DecisionGuard is a mobile-first financial copilot for small Bangladeshi businesses that sell on credit (*baki*), receive most payments through bKash and Nagad, and keep paper or spreadsheet ledgers. It addresses the three parts of Topic E: it matches financing options by their true annual cost, forecasts cash flow and raises tiered alerts before a shortfall, and produces a business financial analysis with statements, ratios, scenario comparison and a decision stability score.

The system is built on one rule: **code computes, the model narrates**. Every figure a user sees is produced by a deterministic engine in exact decimal or integer arithmetic and carries its provenance (formula identifier, version, input facts and source rows). A language model may plan a request and phrase an answer, but every number it states is recomputed independently before it is shown, and the copilot abstains when data, verification or agreement is insufficient. Only anonymised text ever reaches a language model. Every action has a permission tier, and anything that commits the business requires the owner's approval with a fresh one-time code; no component moves money. Settled liquidity-mesh rounds, Trust Passport versions and the daily audit-chain heads are sealed with salted SHA-256 commitments that can be anchored on the Ethereum Sepolia testnet.

The backend is a FastAPI application with pure-function engines over 58 frozen Pydantic contracts; the client is a React progressive web app in English, Bangla and Chinese. At the time of writing the repository holds 2,605 offline tests, the offline evaluation harness passes 62 of 62 tasks, and the cash-flow forecaster has been backtested walk-forward on synthetic shops and on two real public datasets, one of them from a Bangladeshi retailer. Limitations are stated explicitly in Section 8.

Contents

1. Introduction and problem

2. Design principles
3. System architecture
4. Core algorithms
5. Security design
6. Evaluation and testing
7. Deployment
8. Limitations and future work
9. References
10. Appendix A. Component and source map
11. Appendix B. Data sources and licences
12. Appendix C. Reproducing the worked examples

---

1. Introduction and problem

1.1 Problem statement

A typical small Bangladeshi shop sells on credit, takes most payments through the bKash and Nagad mobile wallets, keeps a paper ledger, and borrows informally because it cannot show a lender a clean record. Its owner needs three things from a copilot (Topic E):

1. **Financing:** which financing to take and what it really costs.
2. **Cash flow:** when cash will run short, and what can be done about it in time.
3. **Analysis:** an honest picture of the business.

1.2 Approach

The competition theme is *AI as a Financial Participant*. An agent that acts on money must be right about numbers and must never act alone. The research summarised in `docs/CONTEXT.md` §4 shows why general-purpose language models fail at this task when used directly: they are not calculators, they retrieve the wrong figure, they cannot reliably correct themselves, and their stated confidence is inflated. DecisionGuard therefore separates computation from language: a deterministic core computes every figure, a copilot layer plans and narrates, and a verification layer recomputes every number the copilot states. Section 2 states the design principles that follow from this separation.

1.3 Implementation status

Each component carries one of four status labels:

- **Implemented:** in the submitted code, tested, and reachable through the API and web client.
- **Implemented (reduced scope):** implemented and tested, with named parts planned for later.
- **In progress:** under active implementation; described here for completeness and not counted as evidence.
- **Planned (roadmap):** designed and specified, not part of the submitted code.

Component	Status	Scope notes
Shared contracts (58 frozen models, version 1.2.0), synthetic universe, continuous integration	Implemented	`backend/app/contracts/`, `data/synthetic/`
Typed Fact Ledger, provenance graph, `/api/facts`, `/api/explain`	Implemented	

Component	Status	Scope notes
Data ingestion: bKash, Nagad, bank, ledgers, Bangla normaliser, OCR drafts, preview and commit	Implemented	
Privacy pipeline: detectors, token vault, anonymiser, fail-closed gate, shop-file anonymiser	Implemented	Invoked by the LLM gateway on every model request
Cash-flow engine: forecast, Monte Carlo, gaps, alerts, levers, buffer, backtest, API	Implemented	Email delivery of alerts is planned
Financing engine: true cost, catalogue, eligibility, ranking, lender check, repayment fit on the cash-flow band, API	Implemented	`fit.no_forecast` appears only when the cash-flow engine abstains (too little history)
Business analysis engine: statements, 13 ratios, expected vs reality, scenarios, stability score, API	Implemented	
Liquidity mesh: netting, consent, settlement, residual routing, attestations, anchoring, API	Implemented	
Copilot: LLM gateway, agents, checker, critic, agreement, self-loops, abstention, API	Implemented	
Security layer: authentication, roles, tiers, approvals, audit chain, injection guard, hardening	Implemented	
Self-service shops (hosted demo): sign-up, shop profile, example ledger	Implemented	Demo mode only; data labelled `live`
Trust Passport, sharing consent, verification, state adapter	Implemented (reduced scope)	Sharing records the owner's consent (step-up, expiry, revocation); a lender reads through a consented lender-access account, not a share link. The lender report is planned
Web client (11 screens and a public verification page)	Implemented	Screens without a backend are listed in §3.4
Receivables: aging, risk score, reminder plan, API	Implemented (reduced scope)	Credit limits, dispute log and early-pay calculator are planned
Message drafts: English and Bangla templates, drafts from the reminder plan, owner approval, send log	Implemented (reduced scope)	Sending channels are planned; the owner copies the approved text
End-to-end flows (12 flows, desktop and phone, Playwright with axe)	Implemented	Run in CI on every pull request against a local stack; no hosted run is part of the evidence
Evaluation harness (62 tasks, step logs, evidence bundle)	Implemented (reduced scope)	Mutation score and the nightly audit drill are planned
Machine-learning notebooks	Implemented (reduced scope)	Injection classifier not deployed; GPU training run and market backtest planned
Deployment: Docker, Caddy, deploy with rollback, backups, Render Blueprint	Implemented (reduced scope)	Measurement from mainland China is pending; a second front-end mirror is planned
Market intelligence: price watch, vendor ranking, negotiation support	In progress	Any market price is taken from a dated public source (§4.8)

Component	Status	Scope notes
Nightly audit drill	Planned (roadmap)	

Sources: `docs/PLAN.md` §8, `docs/FEATURES.md` and the submitted code.

1.4 Conventions of this document

- Every number in this document comes from a file in the repository, and the source path is given next to it. Worked examples were computed for this document by running the repository's own engines on the synthetic data; the commands are listed in Appendix C.
- Test counts are taken from `docs/submission/evidence/test-counts.md`, which `npm run evidence` regenerates.
- Amounts are in Bangladeshi Taka, written "Tk". Some worked examples keep the lakh grouping (Tk 48,51,838.00) in which the application displays them.
- **Data notice.** All business data shipped with the repository, and all data in the submitted evidence, is synthetic and labelled `data_mode: "synthetic"`. No real shop data, real users or live language-model traces are included or claimed. Shops that visitors create through self-service sign-up on the hosted demonstration (§7.2) hold whatever their users enter and are labelled `live`; they are not part of the evidence.

---

2. Design principles

DecisionGuard is built around five principles. Each is enforced by code, not by convention.

#	Principle	How the code enforces it
1	**Code computes, the model narrates.** No number shown to a user comes from a language model.	Every figure is a `Figure` contract with mandatory `Provenance` (`formula_id`, version, input fact identifiers, source rows, parameters): `backend/app/contracts/facts.py`. Money is `Decimal`; floating-point values are refused by the `Money` validator (`backend/app/contracts/common.py`). Every number the copilot states is recomputed by the checker before it is shown (`backend/app/agents/checker.py`, §4.6).
2	**Typed facts only.** Engines read the Typed Fact Ledger, never free text.	Engines are pure functions over frozen Pydantic contracts. Fact lookups are exact field-for-field matches; there is no fuzzy or vector lookup (`backend/app/facts/store.py`).
3	**Only anonymised data leaves the server.**	Detectors, token vault, anonymiser and a fail-closed gate (`backend/app/privacy/`). The only path to a language model is `backend/app/llm/gateway.py`, which runs the gate on the messages and the context and refuses the request if any detector fires. The only other optional outbound call is the Sepolia RPC, which carries a 32-byte hash.
4	**Every action has a permission tier** (READ, SUGGEST, DRAFT, EXECUTE, DENY). EXECUTE requires owner approval with a step-up code. Money never moves without a human.	One policy table (`backend/app/security/policy.py`); approvals with a fresh TOTP code, 24-hour expiry and a policy re-check at execution (`backend/app/security/approvals.py`). No executor that moves money exists.
5	**Prefer abstaining to guessing.**	Missing eligibility facts yield `not_on_record`, never a pass (`engines/financing/eligibility.py`); the cash API answers 422 with what is missing when it has under 8 weeks of history (`api/cash.py`); residual-gap routing abstains when nothing is eligible (`engines/mesh/residual.py`); ratios that cannot be computed honestly are empty with a reason (`engines/analysis/ratios.py`); the copilot abstains on short

#	Principle	How the code enforces it
		data, failed checks, low agreement or a noise flip, and logs the abstention (`agents/abstention.py`).

Two further principles govern data labelling:

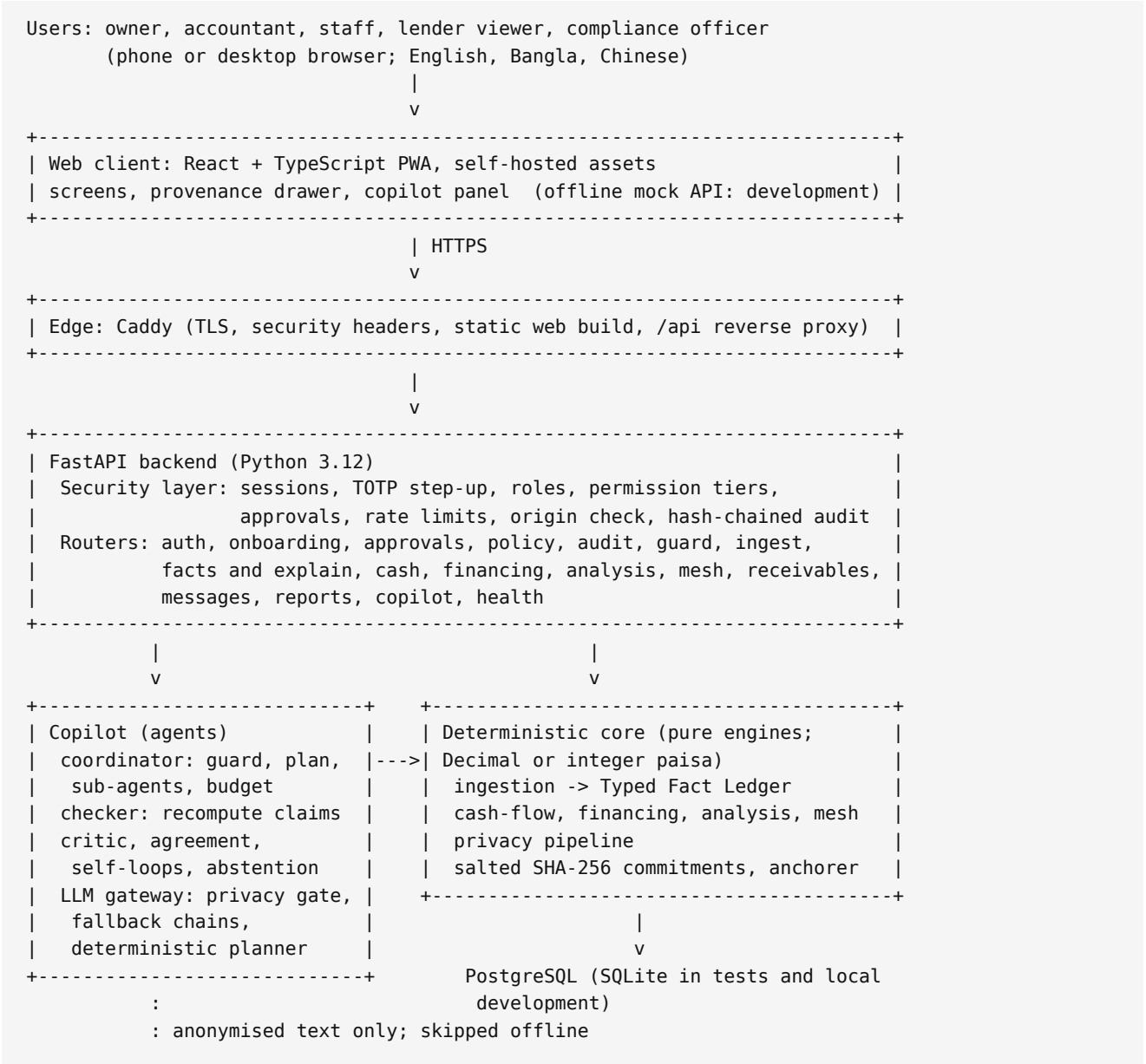
- **Principle 6: synthetic data is labelled synthetic.** Every synthetic record carries `data_mode: "synthetic"`, and the web client shows a synthetic-data banner in demo mode.
- **Principle 7: the most cautious label wins.** When a figure combines inputs with different data modes, it takes the most cautious label (`combined_data_mode` in `backend/app/facts/provenance.py`).

---

3. System architecture

3.1 Layered overview

The system has five layers: the web client, an edge proxy, the API with its security layer, the copilot, and a deterministic core of pure engines over a relational store. A scheduled worker and two optional external services (language models and the Sepolia testnet) complete the picture.



```
          v
Hosted models (Gemini, Groq) or a local Ollama model

Worker (APScheduler, Asia/Dhaka): alert scan 07:30, analysis snapshot freeze,
                                  audit-head anchor 23:55
Ethereum Sepolia (optional): 32-byte digests only, when CHAIN_MODE=sepolia
                             and a funded wallet key is configured
```

Dependencies inside the core run in one direction: the analysis engine uses the cash-flow engine; the cash-flow engine's levers call the financing and mesh engines; the mesh engine's residual routing calls the financing engine; the cash-flow and analysis engines record their figures in the fact ledger. Ingestion passes through the privacy detectors. The security layer writes every sensitive step to the audit chain, whose daily heads, together with settled netting rounds, are sealed by the commitment module.

3.2 Components

Component	Path	Role
Contracts	`backend/app/contracts/`	58 frozen Pydantic models, version 1.2.0; unknown fields rejected; exported to `docs/api/contracts.json` and generated into `web/src/api/contracts.ts` (CI fails on drift)
Application factory	`backend/app/main.py`	Discovers every `app/api/<name>.py` router and every `app/<area>/app_hooks.py`; CORS is added last so that it is the outermost layer; interactive API documentation only in development and test
Engines	`backend/app/engines/{cash,financing,analysis,mesh,receivables}/`	Pure functions: contracts in, contracts out; no database or network access
Copilot	`backend/app/agents/`, `backend/app/llm/`	Coordinator (`agents/orchestrator.py`), tool registry, claims, checker, critic, agreement, self-loops, stakes, budget, abstention; LLM gateway and providers
Fact ledger	`backend/app/facts/`	Store, metric registry, provenance graph, masking hook
Ingestion	`backend/app/ingest/`	File parsers, normaliser, OCR drafts, preview and commit
Privacy	`backend/app/privacy/`	Detectors, vault, anonymiser, gate, shop-file anonymiser
Security	`backend/app/security/`	Authentication, roles, policy, approvals, audit, guard, web hardening
Chain	`backend/app/chain/`	Canonical salted commitments; offline, Sepolia

Component	Path	Role
		and fallback anchorers
Reports	`backend/app/reports/`	Trust Passport, scoped sharing, verification, state adapter
Worker	`backend/app/worker.py`	Discovers `JOBS` in `app/**/jobs.py` and runs them on APScheduler cron (Asia/Dhaka): alert scan, analysis snapshot freeze, audit-head anchor
Web client	`web/src/`	11 screens and a public verification page, provenance drawer, English/Bangla/Chinese, PWA

3.3 Request lifecycle

The following sequence traces one typical interaction end to end.

1. **Ingest.** The owner uploads a bKash statement. `POST /api/ingest/preview` detects the format, normalises amounts and dates, clusters counterparties into opaque `PARTY_` tokens, flags duplicates and returns a preview with a keyed digest. Nothing is written to the ledger yet.
2. **Commit.** `POST /api/ingest/commit` must name the same digest. Only confirmed draft rows (for example OCR lines) are committed. The tenant's ledger revision increases.
3. **Facts.** Ledger metrics (`closing_balance`, `dso` and others) are computed by registered formulas and stored as `Fact` rows that cite their inputs and source rows. A figure computed against an older ledger revision is never served as current.
4. **Engines.** `GET /api/cash/alerts` runs the forecaster, the Monte Carlo simulation, gap detection, alert tiers and levers. The levers call the financing engine (standby loan, early-pay check) and the mesh engine (netting). Each output number is a `Figure` recorded in the ledger.
5. **Explain.** Any number on screen opens `GET /api/explain/{figure_id}`: the formula, the inputs with their values, and the source rows, masked unless the caller holds `source_rows:read` for that tenant.
6. **Act.** An action card (for example "send the application to the lender") is EXECUTE tier. It goes to `POST /api/approvals`; the owner approves with a fresh TOTP code; execution re-checks the policy and the card's SHA-256. Every step is written to the audit chain.
7. **Anchor.** Each night the worker commits every tenant's audit-chain head to one digest and anchors it (on Sepolia when configured, otherwise offline and labelled `offline`; §4.5).

3.4 Web client and API modes

The web client calls the API only through `web/src/api/client.ts`. `web/src/api/apiMode.ts` defines three modes: `live` (real backend only; the production default), `mock` (offline fixtures in `web/src/mocks`, always labelled in the banner) and `auto` (development default: real backend first, mock only for routes whose router is absent).

Served by the live backend	Not served by the backend
Sign-in, TOTP, self-service shop sign-up and profile, team and lender access, approvals inbox, audit verification	Notification preferences in Settings (the card is hidden when the route is absent)
Upload: preview, column mapping, commit	
Cash: forecast, gaps, alerts, what-if	

Served by the live backend	Not served by the backend
Financing: match, lender check, application card	
Analysis: statements, ratios, expected vs reality, compare decisions, stability after Compare	
Mesh: obligations, propose, approve with step-up, verify	
Copilot panel and abstention log	
Receivables: aging, risk, reminder ladder	
Messages: drafts, owner approval with step-up, copy and send log	
Reports: Trust Passport versions, sharing consent, public verification page	
Provenance drawer (`/api/explain`)	

The market screens (price watch, vendor ranking, negotiation) are in progress (§4.8). Screens without a backend run only on the mock, which the banner labels; they show how the finished product is intended to work and are not evidence of backend functionality. The demonstration video uses live screens only (`docs/submission/video-script.md`).

---

4. Core algorithms

This section describes each engine: its inputs, its algorithm, its abstention behaviour, and, where useful, a worked example computed with the repository's own code on the synthetic data.

4.1 Data ingestion and the Typed Fact Ledger

4.1.1 Ingestion

Files: `backend/app/ingest/`.

- **Formats.** bKash and Nagad statements, generic bank CSV, ledger CSV/XLSX with a column mapping step, and ledger photographs. Detection matches header names (English, Bangla, Banglish) exactly after normalisation; preamble lines above the table are skipped (`formats.py`).
- **Normaliser** (`normalise.py`). Handles Bangla digits; Tk/৳/BDT/টাকা markers; lakh grouping and the words লাখ/কোটি/হাজার; month names in three scripts; Bengali calendar dates; and day/month order, decided per file from evidence in that file. A value that cannot be read with certainty raises an error with a reason key and becomes an unparsed row or a draft, never a guess.
- **OCR** (`ocr.py`). Local Tesseract with `ben+eng`; the photograph never leaves the server. Each line becomes a draft with a confidence; low-confidence drafts are flagged and are never committed unless the owner ticks them.
- **Preview and commit** (`preview.py`). The preview stores an HMAC-SHA256 of the uploaded bytes (key derived from the server secret and bound to the tenant). The commit must present the same digest. Duplicates are checked at preview and again at commit. Uploads are capped at 10 MB; previews expire after 24 hours.

4.1.2 Typed Fact Ledger and provenance

Files: `backend/app/facts/{store,registry,provenance,masking}.py`.

- **Fact.** `{id, tenant_id, metric, value: Decimal, unit, period, entity_token, provenance, source_refs, data_mode}`. A lookup `(tenant, metric, period, entity)` must match exactly: a metric name differing in case, or a period one day off, is a miss, never a near match.
- **Metric registry.** 17 named, versioned definitions, each with a `formula_id`, a human-readable expression and a pure function: `cash_in`, `cash_out`, `cash_sales`, `credit_sales`, `credit_purchases`, `purchases`, `operating_expenses`, `receivables`, `payables`, `opening_balance`, `net_sales`, `net_cash_flow`, `closing_balance`, `gross_profit`, `gross_margin`, `dso`, `dpo`. A released definition never changes; a new rule is a new version, so old figures still explain themselves with the old expression. DSO and DPO are on a credit basis (`dso = receivables / credit_sales x days_in_period`) because cash sales never wait.
- **Provenance graph.** `compute()` stores each result with the closure of inputs beneath it, cited as `txn:<id>`, `inv:<id>`, `bill:<id>` or `fact:<id>`. Engines outside the ledger record their own figures with `record_computed()` and `record_figure()`. The cash API records every figure it returns, with the `closing_balance` fact as its cited input (`backend/app/api/cash.py`, class `Recorder`).
- **Staleness.** Every change to a tenant's ledger increments a revision. Computed facts record the revision against which they were confirmed, so a figure that may predate newer rows is recomputed (or reported stale), never served as current.
- **Masking.** Source rows pass through a single masking hook for every caller. The installed policy (`backend/app/security/roles.py`, `policy_masker`) shows rows only to a same-tenant caller holding `source_rows:read`; other callers receive keyed, non-reversible references.

4.2 Cash-flow forecasting and alerts

Files: `backend/app/engines/cash/`. API: `backend/app/api/cash.py`. Feature references: `docs/FEATURES.md` Q1, Q3.

4.2.1 Baseline forecast

Files: `baseline.py`, `sales.py`, `delays.py`. For each of the next 91 days:

```
balance(day) = opening balance + expected inflows - expected outflows

sales(day)        = level x weekday factor x month-end factor x festival factor
collections       = sum over open invoices of owed x P(customer pays on that day)
new credit sales  = recent credit-sales rate x share of invoices paid within n days of issue
outflows          = recurring obligations on their schedule
                  + open bills on the day this business usually pays that supplier
                  + new bills, cash purchases and running costs at their recent daily rate
```

All factors are estimated from the shop's own history up to the forecast date only: weekday and month-end factors from the last 26 weeks; the level from the last 8 weeks; festival factors (Eid-ul-Fitr, Eid-ul-Adha, Pohela Boishakh; `data/public/calendar/bd_festivals.yaml`) from earlier occurrences, with a run-up of 10 days and 3 days after. A factor without evidence stays at 1. Payment delays are each customer's own `paid date - due date` history (the tenant-wide distribution when a customer has fewer than 3 settled invoices), conditioned on the invoice still being unpaid today. If no observed delay remains, the invoice contributes nothing: the forecast prefers under-counting cash to inventing it. Disputed invoices are not expected to be paid.

A machine-learning delay model was evaluated and not adopted. On a time split of the synthetic data, LightGBM scored a mean absolute error of 2.92 days against 2.83 for the per-customer history, and the adoption rule required a 5% improvement (source: `ml/artifacts/payment_delay_metrics.json`).

4.2.2 Monte Carlo simulation

File: `simulate.py`. 1,000 paths, seed 20261020, NumPy PCG64. Amounts are held in integer paisa and ratios in integer basis points, so every band is exact and reproducible:

- **Sales noise:** the shop's own observed-to-expected ratios on past ordinary days, resampled in 28-day circular blocks (a slow fortnight stays slow).
- **Collections:** one lateness draw per customer per path, applied to all of that customer's invoices (a slow payer is slow everywhere).
- **Costs and new bills:** past days' actual spending replayed in 28-day blocks, so one-off shocks recur as often and as large as they did.
- **Obligations and open bills:** treated as certain.

Percentiles are nearest-rank order statistics (P10 is the 100th lowest of 1,000), so `P10 <= P50 <= P90` holds by construction. A what-if or a lever replays the **same draws** (common random numbers): the difference between two runs is the effect of the change, never sampling noise.

4.2.3 Buffer target and gap detection

Buffer target (`buffer.py`): the 95th percentile, over paths, of the deepest dip below the likely (P50) line. Keeping P50 at or above it keeps the shop out of the red in 19 of 20 paths.

Gaps (`gaps.py`): a *risk day* has more than 5% of paths below the buffer; a *gap* is a run of risk days in which more than 20% of paths dip. For each gap the engine reports the first breach, the worst day, the probability (paths below the buffer on at least one gap day), the expected shortfall `mean over paths of max(0, buffer - lowest balance in the gap)`, the lead time, and the three largest drivers in Tk (bills, obligations, late customers, sales below norm). Five leading indicators fire from the ledger before any crossing: DSO rising, a top customer slowing, stock building, sales below the seasonal norm, and payroll or EMI at risk.

4.2.4 Alert tiers

File: `alerts.py`, as specified in `docs/FEATURES.md` Q1. "P within a-b" denotes the share of paths below the buffer on at least one gap day between day a and day b.

Tier	Trigger (any one)	Delivery (design)
Critical	P within 1-7 > 20%; P within 1-21 > 60%; a salary or EMI day in the gap with P50 < 0	in-app + email, pinned on the Today screen
Warning	P within 8-21 > 20%; P within 22-45 > 50%	in-app + email
Watch	P within 22-90 > 20%	in-app + weekly digest

To limit alert fatigue, a repeat alert is sent only if it escalates, or if the forecast moved by more than 3 days or 10% of the shortfall, and never inside the cooldown (24 hours for Critical, 3 days for Warning, 7 days for Watch). In-app delivery is implemented and the daily scan job records deliveries; email delivery is planned.

4.2.5 Levers

File: `levers.py`. Each lever is a scenario replayed on the gap's own draws; its effect is `shortfall without - shortfall with` on the same paths. Only levers that reduce the gap are offered, ranked cheapest first: re-time a payable within its terms, stagger a purchase week, ask for an advance, offer a 2% early-pay discount (only if its annualised cost is below the cheapest eligible loan), net circular debt through the mesh, or arrange a standby loan (financing-engine ranking, sized to cover the gap in 19 of 20 paths). Each lever becomes an action card with its policy tier.

4.2.6 Abstention

With fewer than 8 weeks of transactions, or no opening balance on record, the cash API answers 422 with an `ApiError` naming what is missing (`api/cash.py`, `MIN_HISTORY_DAYS = 56`).

4.2.7 Worked example

Synthetic grocery shop `sme_grocery`, as of 2026-09-30, computed for this document with the repository's engines (Appendix C):

Quantity	Value
Opening balance	Tk 427,842.00
Buffer target (P95 dip)	Tk 296,481.07
Band on 2026-12-30 (day 91): P10 / P50 / P90	Tk 379,696.79 / 511,060.52 / 638,980.00
Plain forecast	no gap; runway = full 91-day horizon
What-if: sales -10%, every customer pays 7 days later (same draws)	4 gaps
First gap: days 4-13, first breach day 10	P = 0.2720, expected shortfall Tk 11,901.90, tier **Warning** (P within 8-21 > 20%)
Its top drivers	two supplier bills of Tk 82,618.00 each, an obligation of Tk 35,000.00
Best lever: stagger a purchase week	cost Tk 0, shortfall reduced by Tk 7,602.01, P after = 0.0960, EXECUTE card
Standby loan lever	cost Tk 2,697.69, shortfall reduced by Tk 4,455.14, DRAFT card
Last gap: days 29-91	P = 0.9770, expected shortfall Tk 159,197.94, tier Warning (P within 22-45 > 50%)

The forecaster's walk-forward backtests, on synthetic shops and on real public data, are reported in §6.4.

4.3 Financing matching

Files: `backend/app/engines/financing/`. API: `backend/app/api/financing.py`.

4.3.1 True cost

File: `cost.py`. Every option is converted into dated cash flows from the borrower's side:

```
day 0      + principal - upfront fees - compulsory deposit
later      - instalments, periodic fees, maturity fees; + deposit returned at maturity

EAR = the r that solves   sum_i  cf_i / (1 + r)^(day_i / 365)  =  0
```

The rate is found by bisection in `Decimal` at 34 digits of precision (the bracket is doubled until the present value turns positive, followed by up to 200 halvings to a width of 1e-12). Bisection is slower than Newton's method but always converges and is deterministic. The engine supports the rate conventions that Bangladeshi lenders publish: reducing balance (EMI), flat (interest on the original principal), daily debit (flat interest repaid in equal daily debits, as nano loans do), discount (interest taken up front) and none (owner top-up, netting). The early-pay discount used by the cash-flow levers is annualised as `(1 / (1 - d))^(365 / days_early) - 1`.

Worked example: Tk 50,000 for 180 days (computed with `true_cost()` on rows of `data/public/financing/bd.yaml`):

Product (catalogue id)	Published rate (basis)	Structure, fees	EAR	Total cost
Bangladesh Bank CMSME WC refinance (`bd-bb-cmsme-wc-refinance-2026`)	9.00% (regulatory cap)	reducing, 0.5% upfront	11.29%	Tk 1,570.67
bKash digital loan, City Bank (`bd-citybank-bkash-loan`)	9.00% (last published)	daily debit, 0.575% upfront	22.15%	Tk 2,506.68
Prime Chalti, Prime Bank (`bd-primebank-chalti`)	15.00% (rate-sheet upper)	reducing, 0.5% upfront	18.11%	Tk 2,460.14
PKSF microenterprise (`bd-pksf-microenterprise`)	24.00% (MRA cap)	reducing, no fee	26.82%	Tk 3,557.74

The same 9% headline rate costs 11.29% a year as a reducing-balance loan and 22.15% as a daily-debit loan, because daily repayment halves the average balance while interest is charged on the full principal. Likewise, a "2% off if paid 10 days early" discount costs 109.05% a year, and 2% for 30 days costs 27.86% (`early_pay_discount_cost`), which is why the levers offer a discount only when it is cheaper than borrowing.

4.3.2 Eligibility

File: `eligibility.py`. Each published rule (for example `business_age_months >= 36` or `trade_licence exists`) returns `pass`, `fail` or **`not_on_record`** when the fact is missing. A missing fact never passes. Amount, tenor and purpose limits can only fail. The overall status is fail if any rule fails; otherwise `not_on_record` if any fact is missing; otherwise pass. The engine also lists exactly which facts the owner could add (`missing_facts`). In the synthetic profiles, `bkash_account_active`, `no_default_record`, `owner_is_woman`, `enterprise_closed` and `collateral_available` are not on record, so products that require them return `not_on_record`.

4.3.3 Repayment fit

File: `fit.py`. Given the cash-flow engine's band, each day's balance is approximated as normal with mean P50 and spread `(P90 - P10) / 2.5631`; the option's cumulative cash flows are added, and the highest daily probability of falling below the buffer is reported. This is a lower bound on the probability of any breach and is labelled as an approximation.

4.3.4 Ranking

File: `ranking.py`. Score from 0 to 100:

```
score = 0.45 cost + 0.25 fit + 0.15 approval + 0.10 speed + 0.05 lender
cost     = max(0, 100 - 2.5 x EAR%)          (4% scores 90; 40% or more scores 0)
fit      = 100 x (1 - P(balance < buffer))
approval = 100 x heuristic by product kind, x 0.6 if facts are missing, x 0.5 if collateral is
needed and not on record
speed    = max(0, 100 - 3 x typical days to cash)
lender   = lender score (60 with no track record)
```

Ineligible options remain in the list with their reasons but score 0 and always rank below every option that might work. Ties are broken on lower EAR, then product identifier, so the order is deterministic. The approval and speed tables are documented assumptions shown to users, not lender data.

4.3.5 Lender check

File: `lenders.py`, `docs/FEATURES.md` N20. Computed from verified track-record events:

```
score = 100 - 50 x (hidden-fee incidents / n) - 30 x (disputes / n)
            - 4 x max(0, mean(realised EAR - quoted EAR)) - 15 if licence unknown
score = 0 if the licence is not found or revoked;   score = 60 with no events (cold start)
```

4.3.6 Application card

File: `application.py`. The best passing option becomes a `submit_loan_application` card: EXECUTE tier, owner approval with step-up, 24-hour expiry. Its payload holds only the facts on record, the facts still missing, the EAR, the rate basis and the source URL. Drafting (`draft_loan_application`) is DRAFT tier.

4.3.7 Current scope of the financing API

As stated in `backend/app/api/financing.py`: repayment fit uses the tenant's cash-flow forecast band, and options carry `fit.no_forecast` only when the cash-flow engine abstains for lack of history; no lender yet has a stored track record; and no licence registry is loaded, so licence status is reported as `unknown`, never assumed. The non-loan options in `nonloan.py` (owner top-up, supplier term extension, buyer early pay, mesh netting) are priced by the same engine but reach users through the cash-flow levers, not through `/financing/match`.

4.4 Business financial analysis

Files: `backend/app/engines/analysis/`. API: `backend/app/api/analysis.py`. Feature references: `docs/FEATURES.md` N1, N4, N10, N12, N16.

4.4.1 Statements

File: `statements.py`. Statements are on a cash basis: income is money received and costs are money paid. Every transaction falls in exactly one line, so the statements reconcile by construction:

```
net_profit + loan flows + owner flows + transfers = cash_in - cash_out = net cash flow
closing cash = opening cash + net cash flow;     total assets = total liabilities + net position
```

Stock and loans appear only when a stock count or loan statement is on record; otherwise they are omitted and named in `missing`.

4.4.2 Ratios

File: `ratios.py`. 13 ratios, each a figure over statement lines with provenance: gross and net margin, current and quick ratio, DSCR, interest cover, DSO, DPO, DIO, cash conversion cycle, burn, runway and top-customer share. A ratio that cannot be computed honestly (a fact not on record, a zero denominator, no debt) is empty with a reason key such as `ratio.missing.stock_value`. Interpretations come from named thresholds, never from a language model.

4.4.3 Expected versus reality

Files: `variance.py`, `store.py`. Each week and quarter, the cash-flow engine's baseline, fitted only on rows dated up to that day, is frozen as an immutable snapshot (a second write is refused). When the period ends, seven cash-flow lines and two totals are compared with what happened and ranked by the size of the difference. Accuracy is 1 minus the weighted absolute percentage error. Recent checks become a sales correction and a cost correction (credibility-weighted towards 1 while there are few), which are passed to the cash-flow engine as a what-if. The nightly job `analysis.snapshot_freeze` takes the snapshots.

4.4.4 Scenario comparison

File: `scenarios.py`. The cash-flow engine's 1,000 seeded paths are split into 40 macro states drawn by Latin hypercube sampling: a sales level shift (standard deviation 8%), input prices (standard deviation 5%), customers paying 0, 7 or 14 days later (70/20/10% of states), rates up to +2 points, and a one-off shock in 10% of states. Six decisions (do nothing, take the best eligible loan from the financing engine, offer an early-pay discount, re-time a supplier, delay a purchase, net through the mesh) are replayed on the same paths and states. For each decision the engine reports the expected outcome, the mean of the worst 5% of outcomes, and P(buffer breach). The selection rule: among decisions within 5 points of the lowest breach risk, choose the highest (expected + worst 5%) / 2; ties go to the cheaper decision.

4.4.5 Decision stability score

File: `stability.py`. A recommendation is re-made under 10 variations: 6 economic stresses, under which the advice may legitimately change (the largest debtor pays 30 days late, sales -10% and -20%, inputs +8%, rates +2 points, a surprise bill of one month's fixed costs due in 7 days), and 4 noise variations, under which it must not (rows shuffled, parties renamed, question reworded, dates written another way). The score is the share of variations in which the choice held; `what_would_flip` names the stresses that changed it; any noise flip marks the advice unreliable. The noise check has already exposed a real defect: the cash-flow engine broke ties between equally owed customers on their token, so renaming two customers swapped their lateness draws. The engine now breaks such ties on record identifiers (`tests/cash/test_names.py`).

4.4.6 Worked example

Synthetic hardware shop `sme_hardware`, quarter July to September 2026, computed for this document through `/api/analysis/*` on the synthetic universe (Appendix C):

Statement line	Value
Revenue (cash sales Tk 33,81,248 + collections Tk 14,70,590)	Tk 48,51,838.00
Purchases	Tk 37,81,353.00
Gross profit	Tk 10,70,485.00
Operating expenses (rent, salaries, utilities)	Tk 4,93,602.00
Net profit (cash basis)	Tk 5,76,883.00
Net cash flow = 5,76,883 - loan repayments 1,26,000 - owner drawings 3,18,599	Tk 1,32,284.00
Closing cash = opening Tk 7,96,734 + 1,32,284	Tk 9,29,018.00 (equals the cash forecast's opening balance)
Named as missing	stock value, loan outstanding

Ratios: gross margin 22.1%, net margin 11.9%, quick ratio 0.91 ("short of current cover"), DSCR 4.58 ("comfortable"), DSO 40.7 days, DPO 33.6 days, top-customer share 4.2%. The current ratio, DIO and the cash conversion cycle are empty (`ratio.missing.stock_value`); interest cover is empty (`ratio.missing.interest_expense`).

Decision comparison (buffer = the cash-flow engine's target of Tk 9,19,664.57; 1,000 paths, seed 20261020):

Decision	Expected outcome	Worst 5%	P(buffer breach)
Do nothing	Tk 16,19,493.02	Tk 4,62,756.51	0.9390

Decision	Expected outcome	Worst 5%	P(buffer breach)
Take the best eligible loan (paid out 30 Oct)	Tk 16,16,268.61	Tk 4,59,455.33	0.9240
Offer 2% off the open invoice with the most cash still expected (pay by 2 Oct)	**Tk 16,11,893.02**	**Tk 4,55,156.51**	**0.5190**
Delay a purchase week	Tk 16,19,493.02	Tk 4,62,756.51	0.9380
Net through the mesh	Tk 16,19,493.02	Tk 4,62,756.51	0.8310
Re-time a supplier	skipped: the shop pays no open bill before its due date		

The discount costs exactly Tk 7,600 (2% of a Tk 3,80,000 invoice) and reduces the breach risk from 93.9% to 51.9%. The loan barely helps because it can only pay out on 30 October, after the gap begins. Stability is 0.80 (8 of 10 variations held); the choice flips under sales -20% and under a surprise bill; all four noise variations held, so the advice is not marked unreliable.

4.5 Liquidity mesh and on-chain anchoring

Files: `backend/app/engines/mesh/`, `backend/app/chain/`. API: `backend/app/api/mesh.py`. Feature references: `docs/FEATURES.md` N14, Q4.

4.5.1 Obligation graph

File: `graph.py`. Only open BDT obligations between two opted-in tenants enter a directed graph, from debtor to creditor, with amounts in integer paisa. Groups with circular debt are the strongly connected components of size 2 or more.

4.5.2 Netting by minimum-cost flow

File: `netting.py`. For each group:

```
node demand(t) = sum owed to t - sum t owes          (net position; > 0 receives)
edge d -> c    = capacity: what d owes c;  cost 1 per paisa
minimise total cash moved, subject to every node meeting its demand    (networkx.min_cost_flow)
```

The original debts are themselves a feasible flow, so a solution always exists; the optimum is the original minus the largest circulation that can be cancelled. Settlements run only along existing debts, so no firm pays a firm it never owed. An opt-in mode (`allow_new_links`) lets any net debtor pay any net creditor and reaches the theoretical minimum, the sum of positive net positions. The following invariants are property-tested with Hypothesis on random graphs (`backend/tests/mesh/test_netting.py`): every net position is unchanged; along debts, no edge carries more than was owed; with new links, nobody pays more than their total debt; the net total never exceeds the gross; no payment cycle remains.

Worked example: the synthetic demonstration ring (`data/synthetic/universe/mesh_obligations.csv`; computed with `build_graph` and `propose_rounds`):

Obligation	Debtor -> creditor	Amount
RING-FG-001	garment -> fabric	Tk 450,000.00
RING-GH-001	hardware -> garment	Tk 320,000.00

Obligation	Debtor -> creditor	Amount
RING-HF-001	fabric -> hardware	Tk 380,000.00
Gross circular debt		**Tk 1,150,000.00**
Net positions	fabric +70,000; hardware +60,000; garment -130,000	
Netting along debts	garment pays fabric 130,000; fabric pays hardware 60,000	**Tk 190,000.00 cash**
Netting with new links	garment pays fabric 70,000 and hardware 60,000	**Tk 130,000.00 cash**

Every net position is the same after either settlement (checked in the run).

4.5.3 Consent and settlement

Consent (`consent.py`). Approving a round is EXECUTE tier: every participant must approve with a verified step-up code; one rejection voids the round; a repeated approval has no effect; rounds expire after 48 hours.

Settlement (`settlement.py`). Only an approved, unexpired round whose obligations are all still open is settled. Each participant receives a receipt: what it pays and receives, which obligations were discharged, and `cash saved = debts cleared - cash paid`. The settled record is then sealed and anchored (§4.5.6). Moving money between SMEs can constitute lending or deposit-taking, so real funding is intended to route through licensed lenders (`docs/CONTEXT.md` §3); no money moves in the submitted system.

API (`backend/app/api/mesh.py`). `GET /api/mesh/obligations` (the caller's debts and those inside its netting group), `POST /api/mesh/rounds/propose` (DRAFT tier: minimum-cost netting of the caller's group, along existing debts), `POST /api/mesh/rounds/{id}/approve` (EXECUTE tier: owner with `X-Step-Up-Code`; the last approval settles the round and anchors its salted digest), `POST /api/mesh/rounds/{id}/reject`, and `GET /api/mesh/rounds/{id}/verify`. The salt never leaves the server. Propose, approve, settle and reject are written to the audit chain. The only obligations in the submitted data are the synthetic demonstration ring; an opt-in flow for real firms is planned.

4.5.4 Residual-gap routing

File: `residual.py`. After netting, a participant may still owe more cash than it holds: `gap = max(0, cash_out - cash on hand)`. Cash on hand comes from the fact ledger; a tenant without that fact is treated as having none (fail closed). The gap is priced with the financing engine for the exact amount and bridge tenor; ineligible products are dropped and the lowest **total cost in Tk** wins (ties: EAR, then the financing engine's rank). If the cheapest option is not the financing engine's top balanced rank, it is flagged `cheaper_than_top_ranked`. With nothing eligible, the router abstains (`mesh.residual.no_eligible_option`) and the round cannot settle for that participant until the owner decides.

4.5.5 Counterparty attestations and lender events

Attestations (`attestations.py`, `docs/FEATURES.md` Q4). Debtor and creditor each sign the SHA-256 commitment of the same obligation record (who owes whom, how much, by when). The record is **peer-verified** only if both parties confirmed that exact digest and nobody filed a dispute. Attestation digests use a fixed public salt so that either side can recompute them.

Lender events (`lender_events.py`). A loan becomes a track-record event for the lender check only when it is fully repaid and every repayment obligation is peer-verified. Then `realised APR` is the EAR (same bisection, §4.3.1) of the cash actually received and repaid; `hidden fee` means the borrower received more than Tk 1 less than principal minus disclosed fees; a dispute counts against the lender without using the contested amounts. Unverified loans are skipped with a reason: no record is preferable to an unverified one.

4.5.6 Salted commitments and anchoring

Files: `backend/app/chain/{digest,anchor}.py`.

```
digest = SHA-256( "hishab:v1:" + kind + ":" + salt + ":" + canonical JSON )
canonical JSON = sorted keys, no spaces, decimals as strings;   salt = 32 random bytes (hex)
```

The salt stays off-chain with the record, so nobody can brute-force small amounts or party names from a public hash. Only the 32-byte digest is ever sent to a chain, and `payload_for()` refuses anything that is not a 64-character lowercase hexadecimal digest. The same mechanism seals netting rounds, Trust Passport versions (`backend/app/reports/passport.py`) and the daily audit-chain heads.

The system supports two anchoring modes:

- **Sepolia mode.** When `CHAIN_MODE=sepolia` and both an RPC URL and a funded wallet key are configured, `SepoliaAnchorer` sends a zero-value self-transfer whose data field is exactly the digest (chain id 11155111) and waits for the receipt. It is wrapped in `FallbackAnchorer`: if the node, the wallet or the block wait fails, the digest is anchored offline instead, so an owner's approval never fails because of the testnet. A malformed key keeps the system in offline mode rather than breaking requests.
- **Offline mode.** Without that configuration, `OfflineAnchorer` records the anchor locally. Every such receipt carries the chain label `offline`, and offline anchors are not presented as independent evidence.

Verification recomputes the digest from the record and salt, reads the anchored data (the transaction's data field in Sepolia mode), and reports which of four checks failed: not anchored, not found on chain, receipt differs from chain, or record changed after anchoring. The daily job `security.anchor_audit_heads` (23:55 Dhaka) anchors through the configured mode; settled netting rounds are anchored by the mesh API, and Trust Passport versions when they are issued (`/api/reports/passport`). Tests use a fake web3 object and never contact a node (`backend/tests/mesh/test_sepolia.py`).

The submitted evidence states which mode produced each anchor. The anchors in the current evidence bundle were produced in offline mode and are labelled `offline` (`docs/submission/evidence/INDEX.md`); no Sepolia transaction hash is claimed in this document.

4.6 Copilot and claim verification

Files: `backend/app/llm/`, `backend/app/agents/`. API: `backend/app/api/copilot.py` (`POST /api/copilot/ask`, `GET /api/copilot/today`, `GET /api/copilot/abstentions`). Feature references: `docs/FEATURES.md` N8-N11, N17, P9, P13.

4.6.1 LLM gateway

Files: `llm/gateway.py`, `llm/providers.py`. One OpenAI-compatible client is used for all providers; only the base URL, key and model change. The gateway defines four slots: the *brain* (Gemini Flash); the *voters* (gpt-oss-120b and Qwen, falling back to gpt-oss-20b and Gemma on Groq); the *critic* (the same pool, never the brain's model family); and a *private* slot (a local Ollama model on infrastructure the operator controls). Each request walks its chain: cache, key present, within the system's own per-slot quota (requests per minute and per day, with doubling cooldowns), send, valid JSON. Any failure moves to the next link. **The last link of every chain is a deterministic planner, so the application works with no model at all.** The cache is keyed by model and the hash of the anonymised prompt. Tests replay recorded fixtures (`LLM_MODE=fixtures`).

4.6.2 One question, end to end

The coordinator (`agents/orchestrator.py`) processes each question in ten steps:

1. **Guard.** The question passes the security layer's injection guard before any model sees it; a hit is quarantined and only its hash is audited.
2. **Plan.** The brain proposes intents and tools from the anonymised question. Code repairs the plan (`planning.py`): unknown intents and tools are dropped, dependencies run first, and what-if shocks are read from the owner's words by rules, never by a model.
3. **Dispatch.** One sub-agent per intent runs in parallel threads, sharing a memoised tool runner. A `Budget` caps tool calls, model calls and tokens per run (P13). Tools (`tools.py`, `toolset.py`) have validated parameters and a minimum history, are never EXECUTE tier, and every figure they return must cite facts, rows or parameters: `cash.forecast`, `cash.gaps`, `analysis.scenarios` (cash-flow what-if), `financing.match` (sized to the gap, with the forecast band), `analysis.ratios`, `mesh.propose`.
4. **Compose.** Every number becomes a `Claim{text, value, formula_id, inputs}` rendered by code in English, Bangla or Chinese. The brain only narrates the claim texts. At most three action cards that the caller's role may act on are attached (P9).
5. **Check** (`checker.py`). Each claim must match its figure and state its value exactly as code renders it, and is then recomputed independently:

Source	Recomputed from
Fact-ledger metrics	the cited input facts re-read from the ledger, through the registry *and* through SymPy on the published expression, so the code and the formula a user reads must agree
Cash-flow figures	a freshly drawn simulation with the figure's seed, buffer and scenario, through the engine's own figure and lever functions
Financing costs	the catalogue product's cash flows for the figure's principal and tenor
Mesh netting	the netting round, recomputed from the cited obligations

A mismatch is rejected; a claim with no recomputation path is unverifiable. In either case it is removed from the answer and logged. A narration containing a number that no verified claim states is replaced by the claim texts. The tests inject a wrong number into each of the 26 kinds of stated number. 6. **Stakes** (`stakes.py`). Low stakes = information; medium = drafts, netting, what-ifs; high = borrowing or moving money. Stakes decide how much checking happens before the owner is asked. 7. **Critic** (`critic.py`, medium stakes and above). A model from a family that took no part in the answer argues the case against it, given the verified claims and the risks found by code (approval slower than the gap, unknown eligibility, still below the buffer, netting needs every firm's consent, short history, synthetic data). An objection that states a number no verified claim states, or the absence of any objection, falls back to the worst risk found by code. 8. **Agreement** (`agreement.py`, medium stakes and above). The judgement on which the answer rests goes to two other models, alongside code's own ranking as the first vote. The agreement rate is the share of votes for the winner; the threshold is a majority, 0.66 or unanimity, by stakes. The deterministic rules never vote, so offline or out of quota **no agreement is claimed**. 9. **Self-loop** (`selfloop.py`). The option on which the answer rests is re-made under stress and noise (no variations at low stakes, 2 at medium, 4 at high) and reported with what would flip it. 10. **Abstain** (`abstention.py`). The answer is withheld when no claim recomputed, when any claim was rejected at high stakes, when the voters do not support code's choice at the stakes' threshold, or when the decision flips on noise. Every abstention names what is missing, is stored, and is appended to the audit chain as `copilot.abstain`.

4.6.3 Worked example

Synthetic hardware shop, offline: no model was called, so the deterministic planner planned and narrated, the critic objection is the worst risk found by code, and no agreement is claimed. Computed for this document with `POST /api/copilot/ask` (Appendix C):

Question	Tools run	Claims stated / verified	Action cards	Critic	Stability
"Will I run out of cash in the next month?"	`cash.forecast`, `cash.gaps`	12 / 12	2 x ask a buyer for an advance (EXECUTE)	"Asking a customer to pay early can strain the relationship and may be refused."	0.75: if customers pay 7 days later, netting becomes the better first fix
"How big a loan can I safely take?"	`cash.gaps`, `financing.match`	14 / 14	2 x advance (EXECUTE), 1 x propose a netting round (DRAFT)	"We could not confirm that you qualify for Startup financing ...: the lender will ask for facts DecisionGuard does not hold."	1.00

From the second answer, every figure was verified: a cash gap starting in 5 days with probability 86.70% and expected shortfall Tk 73,460.90; the cheapest fix closes Tk 71,736.88 of it at no cost; the Bangladesh Bank startup fund costs 4.07% a year, Tk 490.28 in total for this need, and leaves a 47.70% probability of still falling below the buffer.

No live-model trace is part of this document. Live answers can be recorded as fixtures and JSONL traces with `python -m app.llm.live_check` (provider keys required); the evidence index lists any that were recorded.

4.7 Privacy pipeline

Files: `backend/app/privacy/`. Feature references: `docs/FEATURES.md` N6, N18, P11. The pipeline runs on every model call.

1. **Detect** (`detectors.py`). Bangladeshi mobile numbers (bKash and Nagad accounts are mobile numbers), NID numbers, payment cards (Luhn check), long account numbers, email addresses, Dhaka addresses and area names, and person and shop names from the party registry and a name list. Bangla digits are recognised everywhere. The detectors fail closed: any standalone run of 10 or more digits that is not a phone number is treated as sensitive.
2. **Tokenise** (`vault.py`). `PERSON_xxxxxxxxxx = HMAC-SHA256(tenant | kind | normalised value)`, truncated, so tokens are stable within a tenant and different across tenants. Originals are stored with AES-256-GCM (random 96-bit nonce; tenant and token as associated data, so a ciphertext copied elsewhere fails to decrypt). HMAC and encryption keys are derived separately from `VAULT_KEY` with HKDF. Restoring an original requires an authoriser; the default refuses every caller.
3. **Anonymise** (`anonymiser.py`). Tokens become neutral labels in order of appearance ("Customer 1", "Firm A"); dates become relative ("month t-2"); place names and rare product terms become `[place]` and `[item]` (a memorisation guard); optionally, amounts are multiplied by a secret factor so that ratios survive and scale does not (P11). `restore()` maps labels back to tokens. Numbers in a model's answer are never restored or trusted.
4. **Gate** (`gate.py`). `assert_safe(payload)` runs every detector on the serialised payload and raises an error if any fires. The error names only kinds and counts, never values.

Within the copilot, the coordinator anonymises the owner's question before the planner sees it, and the narrator and critic receive only verified claim texts (figures, product names, party labels, no personal data; amounts are not scaled for these calls). `prepare_messages()` offers one label set across several messages for callers that need it. The gateway (`backend/app/llm/gateway.py`) runs the gate on the messages *and* the structured context before any provider is tried. If a detector fires, the request is refused and counted, never sent. Model answers return with labels, which are mapped back to tokens; numbers in them are never used (§4.6). For field data, the shop-file anonymiser (`python -m app.privacy.shopfiles`) tokenises CSV/XLSX columns, refuses any file with residual personal data, and writes a counts-only report.

4.8 Supporting modules

- **Receivables** (implemented, reduced scope). Aging buckets, an empirical 0-100 risk score per open invoice with its top 3 factors (monotone in lateness, property-tested), and a reminder ladder (-3, 0, +3, +10, +20 days) that yields one EXECUTE `send_reminder` card per customer (`backend/app/engines/receivables/`, `GET /api/receivables`, `GET /api/receivables/plan`). Credit limits, a dispute log and an early-pay calculator are planned.
- **Message drafts** (implemented, reduced scope). English and Bangla templates filled from typed values only (an unfilled variable is refused), one draft per reminder, owner approval and a send log (`backend/app/messaging/`, `/api/messages/*`). There is no sending channel yet: the owner copies the approved text into their own chat application. Sending channels are planned.
- **Market intelligence** (in progress). Price watch, vendor ranking and negotiation support (`docs/FEATURES.md` N3, N19; `backend/app/engines/market/`) are under implementation. Any market price the system shows is taken from a dated public source: the Trading Corporation of Bangladesh's published daily retail price lists for Dhaka (`data/public/prices/tcb/`), shown as "published on by TCB" and never as live prices. Any vendor quotes in the demonstration shops are synthetic.

---

5. Security design

This section summarises the security design. The full Security Self-Assessment (`docs/submission/security-self-assessment.md`) contains the generated permission-tier table, the STRIDE threat model, the mapping to the OWASP Top 10 for LLM Applications, and the known-risk list. All controls below are implemented in `backend/app/security/` unless another path is given.

Control	Design	Code
Permission tiers	One table decides (a) the tier each action requires and (b) how far each role may take it. EXECUTE means an owner approved this exact card with a fresh TOTP code less than 24 hours ago, and the policy still allows it at execution. A card's own tier can raise the requirement, never lower it, so model-written cards cannot lower their own tier.	`policy.py`
DENY list	`change_bank_details`, `add_payee`, `disable_audit`, `export_raw_pii`: refused for every role, including the owner; no executor can be registered for them.	`policy.py`, `approvals.py`
Approvals	Idempotency keys on request and execution; the approved card's SHA-256 is stored and re-checked; the policy is re-checked at execution. No executor is registered in the submitted system, so an executed card is recorded in the audit chain, nothing is sent and no money moves.	`approvals.py`
Roles	owner, accountant, staff, lender_viewer, compliance. Lender access exists only through the owner's recorded consent: named lender, purpose, scopes, at most 30 days, revocable, created with step-up.	`roles.py`, `team.py`
Authentication	argon2id password hashing; RFC 6238 TOTP, single-use per step; server-side sessions in HttpOnly SameSite=Strict cookies with 30-minute idle and 12-hour absolute limits; 5 failures in 15 minutes lock the account for 15 minutes. Demo accounts and self-service sign-up exist only in demo mode; production refuses demo mode. Sign-up is capped and rate limited.	`passwords.py`, `totp.py`, `sessions.py`, `lockout.py`, `backend/app/api/onboarding.py`
Audit chain	Per-tenant SHA-256 hash chain; ORM and database triggers refuse UPDATE and DELETE; verification finds the first bad link;	`audit.py`, `jobs.py`

Control	Design	Code
	daily heads are anchored (§4.5.6). Entries hold identifiers, counts and digests only.	
Injection guard	Rules for English, Bangla, Banglish and Chinese on normalised text (NFKC, case folding, zero-width and bidirectional controls removed, look-alike letters folded). Hits are quarantined server-side and only their SHA-256 is audited. A classifier slot exists; no classifier is installed (§8.1).	`guard.py`, `guard_rules.py`
Web hardening	CSP, HSTS, nosniff, frame denial, referrer and permissions policies, no-store on API data; per-route token-bucket rate limits; Origin check on state-changing requests; explicit CORS origins (wildcard refused when deployed).	`web.py`
Agent rails	Agent tools are READ to DRAFT only; anything that moves money or commits the business becomes an action card that requires human approval. Each run has a budget of tool calls, model calls and tokens. Model output never supplies a number (the checker recomputes every claim) or a tier.	`backend/app/agents/tools.py`, `budget.py`, `checker.py`
Tenant isolation	Every query is scoped to the principal's tenant; other tenants' records answer 404; a `tenant_id` in a request must match the principal.	`deps.py`, routers
Secrets	Keys exist only in server environment files and CI secrets; `.env` is git-ignored; gitleaks scans the tested commit's full history in CI.	`.github/workflows/ci.yml`

---

6. Evaluation and testing

6.1 Test suite

Test counts, generated on 20 October 2026 by `npm run evidence` (`docs/submission/evidence/test-counts.md`). Every test runs offline: no real language-model call and no network access.

Layer	Area	Tests
Backend (pytest)	agents	346
Backend (pytest)	analysis	132
Backend (pytest)	cash	199
Backend (pytest)	contracts	137
Backend (pytest)	core	37
Backend (pytest)	eval	17
Backend (pytest)	facts	146
Backend (pytest)	financing	150
Backend (pytest)	infra	20
Backend (pytest)	ingest	305
Backend (pytest)	market	59
Backend (pytest)	mesh	113
Backend (pytest)	messaging	11
Backend (pytest)	ml	7

Layer	Area	Tests
Backend (pytest)	onboarding	9
Backend (pytest)	privacy	105
Backend (pytest)	receivables	25
Backend (pytest)	reports	37
Backend (pytest)	security	426
Backend (pytest)	synthetic	87
Backend total		**2,368**
Web units (Vitest)	all	237
Total		**2,605**

The evidence file is authoritative and is regenerated whenever the suite changes.

Layer	Tool	Where
Unit and parametrised scenario tables	pytest	`backend/tests/<area>/`
Property-based invariants	Hypothesis (derandomised in CI, `HYPOTHESIS_PROFILE=ci`)	netting invariants, graph, EAR, eligibility, fit and lender score, Monte Carlo, levers, provenance, deduplication, anonymiser round-trip, vault, contracts
Contract drift	pytest and a CI step	JSON Schema against `docs/api/contracts.json`; the generated `web/src/api/contracts.ts` must not change on regeneration
Web units, i18n completeness, accessibility	Vitest, axe	`web/src/**/*.test.tsx`
End-to-end	Playwright with axe	`web/e2e/journeys/`: 12 flows on desktop and phone, run in CI on every pull request against a local stack. A further check fails if any screen contacts a host other than its own (`web/e2e/journeys/language.spec.ts`)

Offline rule. Tests never call real language models or the network: a session-wide fixture in `backend/tests/conftest.py` makes any non-loopback socket connection fail the test. The Sepolia anchorer is tested against a fake web3 object. The live OCR test runs only where Tesseract `ben+eng` is installed (the API image) and is skipped in CI.

6.2 Continuous integration

`.github/workflows/ci.yml` runs on `main` and on every pull request:

1. **Backend:** ruff, pytest, `pip-audit --strict` on the locked dependencies.
2. **Web:** ESLint and TypeScript, contracts in sync, Vitest, `npm audit --audit-level=high`, build.
3. **Secrets:** gitleaks over the full history of the commit under test (an all-refs scan is performed before the repository is made public).
4. **Docker stack:** build all images, boot the Compose stack, then smoke-test through Caddy: TLS, `/api/health` reporting PostgreSQL, SPA fallback, CSP, HSTS and `X-Frame-Options` headers, and HTTP-to-HTTPS redirect.

`.github/workflows/e2e.yml` runs the end-to-end flows against a local stack on every pull request; the same workflow has a job for a hosted target.

6.3 Evaluation harness

Files: `backend/eval/`; results in `docs/submission/evidence/README.md` and `docs/submission/evidence/eval/offline/`. `npm run eval` runs the real copilot over the synthetic ledger, offline: model calls replay recorded answers, and a prompt with no recording falls through the gateway's chain to the deterministic planner, exactly as when every model is unavailable. Each task writes a per-step JSONL log (guard, plan, each tool call with its figures and formulas, checker, narration, critic, votes, self-loop, and what the owner would see). The harness is not a CI gate.

Suite	Passed
Functional	26/26 (100%)
Adversarial	36/36 (100%)
All tasks	**62/62 (100%)**

Reliability metrics from the same run:

- **Abstention:** 6 of 6 tasks that should abstain did so, all 6 with the expected reason; 0 of 20 answerable tasks abstained.
- **Numbers shown:** 174 claims, all 174 recomputed by the symbolic checker to the same value.
- **Noise (N16):** 0 of 16 irrelevant variations in the self-loops flipped a decision.
- **Agreement (N9):** no vote offline (the voters have no recordings); agreement can only be measured in a live run.
- **Latency per task:** median 51 ms, maximum 2,445 ms.

Interpreting the adversarial result. The adversarial number must be read with care. The rule guard initially caught only 3 of 12 new, unseen attacks (11 of 12 were nevertheless contained, and approval held in all 12). Each attack family's rules were then widened to intent level, with no new false alarms on the 96 benign corpus texts, and the 12 are now caught. Because those attacks were seen while widening the rules, 12/12 is a regression result, not proof of generalisation. On a separate held-out set (24 attacks and 24 innocent look-alikes in four languages, never seen while widening the rules) the rules caught 15 of 24 attacks and wrongly quarantined 10 of 24 innocent questions (`docs/handover/fresh_injection_result.txt`).

6.4 Forecast backtests

6.4.1 Synthetic shops

File: `backtest.py`, `docs/FEATURES.md` N15. Walk-forward: at each origin (every 14 days) the forecaster sees only what was known that day, forecasts 91 days with 1,000 paths, and is scored against what happened. Results on the **synthetic** shops (reproducible with `python -m app.engines.cash.backtest --sme <id>`, which gives identical output on re-run):

SME (synthetic)	MAPE wk 1 ours / naive	MAPE 13 wks ours / naive	P10-P90 coverage	Gap episodes	Hit rate	Median lead	Alerting gaps / did not happen
grocery	9.6% / 15.6%	22.4% / 20.7%	77%	5	100%	76 d	50 / 34
pharmacy	7.9% / 15.5%	16.3% / 17.9%	81%	7	100%	75 d	44 / 27
fabric	9.7% / 21.6%	33.7% / 31.0%	73%	24	100%	84 d	38 / 10
garment	23.2% / 41.2%	30.5% / 51.8%	61%	10	90%	81 d	28 / 0
hardware	7.9% / 18.7%	15.4% / 24.0%	83%	8	100%	87 d	52 / 32

Interpretation: the forecast beats the naive "balance stays the same" baseline in week 1 for every shop, which is the alert window. It loses to the baseline over 13 weeks at the grocery and fabric shops. Alerts are over-cautious at three shops: many alerting gaps did not materialise. These shops were generated by the project's own code, so this backtest validates the harness, not real-world accuracy.

6.4.2 Real public data: UK online retailer

Notebook `ml/notebooks/01_forecast_backtest.py`; results in `ml/artifacts/forecast_backtest.json` and `forecast_backtest_origins.csv`. Data: UCI Online Retail II (CC BY 4.0), a real UK online retailer's 1,067,371 invoice lines converted into 739 days of net cash-in (December 2009 to December 2011). 19 walk-forward origins, 14 days apart, each forecasting 28 days from data before the origin only. Cash gaps are judged on a simple outflow rule (weekly supplier run at 95% of average inflow, rent on the 1st) with the opening balance set one day of sales above the expected low point.

Model	WAPE	Weekly MAPE	80% band covers	Real gaps flagged	False alarms	Lead time
Seasonal naive (same weekday last week)	43.4%	24.8%	71.2%	3 of 5	42.9%	10.7 d
ETS (the method family the engine uses)	36.3%	22.9%	71.8%	3 of 5	28.6%	10.7 d
LightGBM quantile	34.1%	22.2%	54.0%	3 of 5	50.0%	10.7 d

Interpretation: on real data not generated by the project, ETS beats the naive baseline on every error measure and raises fewer false alarms. LightGBM is slightly more accurate, but its 80% band covers only 54% of days, so its warnings are overconfident; ETS is the model the notebook selects. Gaps that were flagged were flagged about 11 days ahead. Only 5 real gaps occurred, so the hit and false-alarm rates are approximate, and this is one retailer, not a Bangladeshi shop.

6.4.3 Real public data: Bangladeshi retailer

Same notebook with `HISHAB_DAILY_PATH`; results in `ml/artifacts/forecast_backtest_bd.json`. Data: a Bangladeshi retailer's daily units sold for one product, 1 January 2013 to 31 December 2017 (Mendeley Data, DOI 10.17632/xwmbk7n3c8.1, CC BY 4.0, copy in `data/public/retail_bd/`), used as a proxy for daily inflow. 103 origins from 2014 onwards, with the same 28-day horizon and gap rule.

Model	WAPE	Weekly MAPE	80% band covers	Real gaps flagged	False alarms	Lead time
Seasonal naive	26.1%	11.1%	77.7%	41 of 45 (91%)	15.5%	17.1 d
ETS (chosen)	19.9%	10.8%	73.0%	37 of 45 (82%)	17.2%	17.8 d
LightGBM quantile	19.9%	9.1%	56.9%	39 of 45 (87%)	15.5%	17.5 d

Interpretation: on real Bangladeshi sales, ETS reduces the daily error by about a quarter relative to the naive baseline, but it does not flag more gaps: the naive forecast caught 4 more of the 45, at a similar false-alarm rate. Better accuracy did not translate into better warnings here, so the alert layer, and not only the forecaster, needs tuning on real shop data. The series covers one product of one shop, measured in units rather than Tk.

6.5 Execution evidence

The sandbox operation log (`docs/submission/evidence/operations/operation-log.md`, regenerated by `scripts/evidence_operations.py`) records 48 API and engine steps end to end through the real FastAPI application: sign-in with TOTP; upload, commit and duplicate re-upload; cash forecast, gaps, alerts and a what-

if; the provenance of a figure; financing match and lender check; the full EXECUTE path (refused without and with a wrong step-up code, approved with a fresh code, executed, idempotent replay); injection attempts in Bangla, English and Chinese; cross-tenant and anonymous calls refused; audit-chain verification and anchoring; and the mesh ring netted, approved, settled, sealed, verified and tamper-detected.

---

7. Deployment

7.1 Topology

- **Containers** (`infra/docker-compose.yml`): `db` (PostgreSQL 16), `api` (FastAPI), `worker` (same image, `python -m app.worker`) and `web` (Caddy serving the built web client and proxying `/api/*` to the API). Secrets come from a server-only environment file, never from git.
- **TLS and headers** (`infra/Caddyfile`): automatic Let's Encrypt certificate for `DEPLOY_DOMAIN`; HSTS, a strict CSP (`default-src 'self'`, no third-party origins), `X-Frame-Options: DENY`, `nosniff`, COOP and compression. The API adds its own headers as well, so a request that bypasses Caddy is still covered (`backend/app/security/web.py`).
- **Region.** Singapore was chosen because the Gemini API is not served in Hong Kong or mainland China but is served in Singapore.

7.2 Hosted demonstration and self-service shops

The hosted demonstration runs at **https://decisionguard-web.onrender.com** on Render's free tier in Singapore, configured by the Blueprint `render.yaml`: a free PostgreSQL database, the API (Docker; it loads the synthetic demonstration shops on start) and the web client as a static site that forwards `/api/*` to the API, so the browser sees one origin and the session cookie stays first-party. Secrets are generated by the hosting platform or entered in its dashboard; none are in the repository. The Blueprint sets `CHAIN_MODE=sepolia` with a public RPC endpoint; the wallet key is entered only in the dashboard, and without it (or if the testnet fails) every anchor is made offline and labelled `offline` (§4.5.6). The Blueprint uses recorded model fixtures by default (`LLM_MODE=fixtures`); live models are enabled by entering provider keys in the dashboard.

Visitors can use the demonstration in two ways:

- **Demonstration shops.** On the sign-in page, one action signs the visitor in to a synthetic shop as owner, accountant, staff member, lender viewer or compliance officer. In demo mode the owner's step-up code is shown on screen. These shops are labelled synthetic.
- **Self-service shops.** "Create your own shop" (`POST /api/onboarding/signup`) creates a separate tenant for the visitor. The owner enters the opening cash balance and the shop facts that lenders check (business age, trade licence, e-TIN, staff) through `/api/onboarding/profile`, downloads an example ledger (`/api/onboarding/template.csv`) or uses their own CSV/Excel file, and uploads it; every screen then runs on the entered numbers. Such a shop is marked `live`, not synthetic, and shows no synthetic banner; each owner-entered value is stored as a raw fact whose source is the owner's entry. Sign-up is available only in demo mode, capped at 500 shops and rate limited. Because the server is public, visitors are asked to enter test or anonymised figures.

7.3 Own-server deployment, continuous deployment and backups

`docs/DEPLOY.md` sets up the same stack on a Linux server with Docker Compose behind Caddy. After CI passes on `main`, `.github/workflows/deploy.yml` and `infra/remote-deploy.sh` ship the exact commit to the server, rebuild, run a health check at `/api/health`, and roll back to the previous release on failure; the workflow does nothing until the server secrets exist. Hourly `pg_dump` backups keep 48 copies (`infra/backup.sh`). A server holding real shop data runs with `DEMO_MODE=false` and is never seeded with demonstration data.

7.4 Reachability from mainland China

The web client loads no third-party resources: there are no Google Fonts, CDNs or analytics, and fonts and assets are bundled. Swagger UI, which loads from a CDN, is disabled outside development (`backend/app/main.py`). An end-to-end test fails if any screen contacts a host other than its own (`web/e2e/journeys/language.spec.ts`). Measurements from mainland nodes are pending.

---

8. Limitations and future work

8.1 Limitations

1. **Data.** All data in the repository and in the evidence is synthetic. The backtest results in §6.4.1 and the worked examples in §4.2, §4.4 and §4.6 come from shops produced by the project's own generator; they test the engines and the harness, not real-world accuracy. Real anonymised shop data is not included: field visits did not yield data in time. Public real data is used instead for the forecast backtests (§6.4.2, §6.4.3) and is never described as data from the project's own shops.
2. **Injection classifier not deployed.** The CPU baseline (multilingual DistilBERT, 4 epochs) reaches AUC 0.6875 on unseen attack families, with precision 0.5714 at the recall-1.0 threshold (source: `ml/artifacts/injection_metrics_cpu_baseline.json`). This is too weak for use, so the guard is rules-only.
3. **The rule guard is proven against known attacks, not unseen ones.** The rules catch all 96 attacks and flag none of the 96 benign texts in the corpus (`backend/tests/security/test_guard.py` over `data/public/injection/injection_corpus.jsonl`), but they were written with sight of that corpus. On 12 new attacks written without reading the rules they caught 3 (the copilot still acted on none, and no approval was granted). The rules were then widened per attack family and now catch those 12 with no new false alarms, which is likewise a regression result. On a fresh held-out set they caught 15 of 24 attacks and wrongly blocked 10 of 24 innocent questions that use the same words. The guard slows an attacker down; the approval rules (owner approval plus a fresh code for anything that leaves DecisionGuard) are what keep money safe.
4. **Repayment fit is an approximation.** Fit is the highest daily breach probability over the forecast, using the tenant's cash-flow band (normal approximation from P10/P90). When the riskiest day falls before the first repayment, every option receives the same fit and fit does not change the order (for the synthetic grocery shop: 0.48% for all options). With too little history the term is dropped and options carry `fit.no_forecast`.
5. **Eligibility encodes only the conditions that could be sourced.** For example, the Bangladesh Bank startup fund is matched on a business age of at most 144 months and a trade licence, so a 6-year-old synthetic shop passes. Real lenders apply further criteria. Approval likelihood and days-to-cash are heuristics and are shown as such.
6. **No lender track record or licence registry.** No verified loans are stored and no licence registry is loaded, so every licence status is `unknown` (`backend/app/api/financing.py`).
7. **On-chain anchoring is evidence only where the evidence says so.** An anchor counts as on-chain evidence only if the evidence index lists a Sepolia transaction; the anchors in the current evidence bundle are offline. Offline anchors are stored locally and are not independent evidence.
8. **The model layer was measured offline.** Every copilot number in this document was produced with no model: the deterministic planner planned and narrated, and no agreement was claimed. With keys configured, free-tier providers impose rate limits and may train on the (anonymised) inputs. Questions outside the planner's intents abstain as out of scope.
9. **Market data is published, not live.** Market prices are taken from TCB's daily Dhaka retail lists as published (10 lists, 10 to 19 October 2026, stored in the repository and not fetched on a schedule), so they are at best one day old; vendor quotes in the demonstration are synthetic. The market module is in progress (§4.8).

10. **Hosting.** The hosted demonstration runs on a free tier: after 15 idle minutes the server sleeps and the first page takes about a minute to load. Reachability from mainland China is unmeasured. The stack is also verified by the CI boot test and runs locally (`npm run dev`) or with Docker Compose (`docs/DEPLOY.md`).
11. **Platform risks** carried from the Security Self-Assessment: in-process rate limits; no phishing-resistant second factor or recovery codes; an owner can be locked out for 15 minutes by someone who knows the email address. Details: `docs/submission/security-self-assessment.md` §9.

8.2 Planned work (roadmap)

Area	Planned item
Cash-flow alerts	Email delivery
Receivables	Credit limits, a dispute log and an early-pay calculator
Messaging	Sending channels (currently the owner copies the approved text)
Reports	The lender report
Market intelligence	Completion of price watch, vendor ranking and negotiation support; a backtest of market alerts; vendor quotes entered by self-service shops
Liquidity mesh	An opt-in flow for real firms; routing of real funding through licensed lenders
Security	A trained injection classifier (GPU training run); a nightly audit drill
Evaluation	Mutation testing; live-model runs that measure agreement
Data	Real anonymised shop data collected in the field, processed with the shop-file anonymiser
Deployment	Reachability measurements from mainland China; a second front-end mirror

---

9. References

Document	Path
Security Self-Assessment (tier table, threat model, OWASP LLM Top 10 mapping, known risks)	`docs/submission/security-self-assessment.md`
Execution evidence (test counts, evaluation results, operation log, freeze record)	`docs/submission/evidence/README.md`, `docs/submission/evidence/INDEX.md`
Feature specification	`docs/FEATURES.md`
Problem context and research summary	`docs/CONTEXT.md`
Project plan and status	`docs/PLAN.md`
Deployment guide	`docs/DEPLOY.md`
Demonstration video script	`docs/submission/video-script.md`
API contracts (JSON Schema)	`docs/api/contracts.json`

Appendix A. Component and source map

Component	Source folder	API router
Shared contracts	`backend/app/contracts/`	
Synthetic universe loader	`backend/app/synthetic/`, generator `data/synthetic/generate.py`	

Component	Source folder	API router
Typed Fact Ledger and provenance	`backend/app/facts/`	`backend/app/api/facts.py`
Data ingestion	`backend/app/ingest/`	`backend/app/api/ingest.py`
Privacy pipeline	`backend/app/privacy/`	
Cash-flow engine	`backend/app/engines/cash/`	`backend/app/api/cash.py`
Receivables engine	`backend/app/engines/receivables/`	`backend/app/api/receivables.py`
Financing engine	`backend/app/engines/financing/`	`backend/app/api/financing.py`
Business analysis engine	`backend/app/engines/analysis/`	`backend/app/api/analysis.py`
Liquidity mesh	`backend/app/engines/mesh/`	`backend/app/api/mesh.py`
On-chain anchoring	`backend/app/chain/`	
Market intelligence (in progress)	`backend/app/engines/market/`	`backend/app/api/market.py`
Copilot (LLM gateway and agents)	`backend/app/llm/`, `backend/app/agents/`	`backend/app/api/copilot.py`
Security layer	`backend/app/security/`	`backend/app/api/auth.py`
Self-service shops	`backend/app/api/onboarding.py`	`backend/app/api/onboarding.py`
Message drafts	`backend/app/messaging/`	`backend/app/api/messaging.py`
Reports and Trust Passport	`backend/app/reports/`	`backend/app/api/reports.py`
Scheduled worker	`backend/app/worker.py`, `backend/app/**/jobs.py`	
Evaluation harness	`backend/eval/`	
Web client	`web/src/`	
End-to-end tests	`web/e2e/`	
Machine-learning notebooks and artefacts	`ml/notebooks/`, `ml/artifacts/`	
Infrastructure and CI	`infra/`, `.github/workflows/`, `render.yaml`	

Appendix B. Data sources and licences

B.1 Synthetic SME universe (labelled synthetic)

`data/synthetic/generate.py` (seed 20261020, deterministic: the same seed produces the same bytes; file hashes in `data/synthetic/universe/manifest.json`) generates five Dhaka SMEs with 18 months of daily data, 2025-04-01 to 2026-09-30 (as-of date 2026-09-30):

SME id	Display name (fictional)	Sector
`sme_grocery`	Mirpur Mudi Store	grocery
`sme_pharmacy`	Dhanmondi Care Pharmacy	pharmacy
`sme_fabric`	Islampur Fabric House	fabric wholesale
`sme_garment`	Ashulia Stitch Works	garment subcontractor
`sme_hardware`	Nawabpur Hardware Traders	hardware

Source: `data/synthetic/universe/<sme>/profile.json`. Each shop has raw files in the formats a real shop would export (bKash, Nagad, bank statement, cash book, baki register, bills register, obligations) plus `transactions_truth.csv` as typed ground truth, so the ingestion parsers are tested on realistic files. The data has weekday, month-end and festival seasonality, credit sales with per-customer delay habits, recurring rent, salaries and EMIs, and the three-firm debt ring of §4.5.2. Every row carries `data_mode = synthetic`, and the web client shows a synthetic-data banner in demo mode.

B.2 Bangladesh financing catalogue (public, sourced)

`data/public/financing/bd.yaml` holds 13 priced products and 5 unpriced ones (counted with `load_catalogue()` and `load_unpriced()`). Every row was checked on 2026-10-18 and carries a `source_url` and a verbatim `evidence_quote`; the loader rejects unknown fields. Each rate has a `rate_basis`: `published`, `rate_sheet_upper` (upper bound of a published range), `regulatory_cap` (for example the Microcredit Regulatory Authority's 24% cap) or `last_published`. The user interface shows the basis next to every rate. Assumptions are stated in the file header (reducing balance unless stated; Bangladesh Bank's 0.5% processing-fee ceiling applied without its Tk cap; trade licence assumed where a page lists no eligibility, labelled "typical").

Products whose provider publishes no rate (for example invoice financing, factoring and two bKash-channel loans) are listed under `unpriced` and shown as "ask the lender". **They are never ranked on a guessed cost.** The regulatory facts behind the catalogue, with quotations, are in `data/public/financing/bd_regulatory.yaml`. A second catalogue, `cn.yaml`, holds 3 products modelled on Chinese digital SME credit (P14). They are illustrative, labelled `is_modelled: true` and shown as "modelled"; they demonstrate that the engine is portable and are not real offers.

B.3 Public datasets and other data

Data	Path	Source and licence
Bangladeshi retailer daily sales, 2013-2017	`data/public/retail_bd/dataset.xlsx`	"Supply Chain Demand Forecasting Dataset of Bangladeshi Retailer", Mendeley Data, DOI 10.17632/xwmbk7n3c8.1, CC BY 4.0; unchanged
UK online retailer transactions, 2009-2011	used by `ml/notebooks/01_forecast_backtest.py`	UCI Machine Learning Repository, Online Retail II, CC BY 4.0
Dhaka daily retail prices (10 lists, 10 to 19 October 2026)	`data/public/prices/tcb/` (URLs in `sources.csv`)	Trading Corporation of Bangladesh (Ministry of Commerce), https://tcb.gov.bd/pages/daily-rmps; public information from a government website with no open-data licence stated; files kept unmodified and cited with URL and publication date; non-commercial demonstration use
Bangladeshi festival calendar	`data/public/calendar/bd_festivals.yaml`	Eid dates are +/- 1 day
Injection corpus	`data/public/injection/injection_corpus.jsonl`	192 synthetic rows (96 attack, 96 benign) in four languages, built by `data/public/injection/build.py`
Financing catalogue and regulatory facts	`data/public/financing/`	Lender and regulator web pages; source URL and verbatim quotation on every row (§B.2)

Appendix C. Reproducing the worked examples

Run from `backend/` with dependencies installed (`npm run setup`). Every run is offline and deterministic (seed 20261020).

Example	Procedure
Backtest table (§6.4.1)	`uv run python -m app.engines.cash.backtest --sme sme_grocery` (likewise for each SME)
Cash forecast and what-if (§4.2.7)	`forecast_baseline(...)` on `load_universe().smes["sme_grocery"]` as of 2026-09-30 with `load_festivals()`; `draw()` then `run()` for the plain run and for `Scenario(sales_change_pct=-10, collection_delay_days=7)` on the same draws; `buffer_target()` of the plain run; `find_gaps()`, `tier_gap()`, `smoothing_levers()` with the Bangladesh catalogue and the demonstration ring
True cost table (§4.3.1)	`true_cost(product, Money.of("50000"), 180)` for each catalogue row; `early_pay_discount_cost(Decimal(2), 10)` and `(..., 30)`
Netting ring (§4.5.2)	`propose_rounds(build_graph(load_universe().mesh_obligations), now=..., allow_new_links=False / True)`
Analysis (§4.4.6)	Seed the universe (`uv run python -m app.facts.seed`), sign in as the hardware owner, then `GET /api/analysis/statements` and `/api/analysis/ratios` with `granularity=quarter&on=2026-09-30`, `POST /api/analysis/scenarios` with `{}`, and `POST /api/analysis/stability` with the returned `recommendation_id`
Copilot (§4.6.3)	Same seeded database, `LLM_MODE=fixtures` and no keys: `POST /api/copilot/ask` with `{"question": "Will I run out of cash in the next month?"}` and `{"question": "How big a loan can I safely take?"}`
Guard regression (§8.1)	`uv run pytest tests/security/test_guard.py -q`

Through the API, the same cash figures are returned by `GET /api/cash/forecast` and `POST /api/cash/whatif` for a signed-in demonstration owner of `sme_grocery` (demo mode). The operation log (`docs/submission/evidence/operations/operation-log.md`, regenerated by `scripts/evidence_operations.py`) records 48 such API and engine steps end to end (§6.5).